

FLASH IDS

A Comprehensive Approach to Intrusion Detection via
Provenance Graph Representation Learning

Technical Report

Prepared By: Yug Sheth and Shrey Sojitra

Student IDs: 2023UCP1967 and 2023UCP1967

Course: Computer and Network Security

Course Coordinator: Ramesh Babu Batulla

Abstract

Modern cyber attacks such as Advanced Persistent Threats (APTs) are increasingly stealthy and difficult to detect using traditional Intrusion Detection Systems (IDS). Conventional IDS approaches rely heavily on signatures or simple anomaly detection methods, which fail against zero-day attacks and sophisticated adversaries. FLASH IDS introduces a provenance graph based intrusion detection framework that leverages semantic encoding, temporal encoding, Graph Neural Networks (GNNs), and embedding recycling to detect malicious system activities efficiently and accurately.

FLASH converts system audit logs into provenance graphs and applies Word2Vec based semantic embeddings together with GraphSAGE based graph representation learning to identify anomalous nodes. To improve scalability and enable real-time detection, FLASH employs an embedding recycling database and a lightweight XGBoost classifier. Experimental results demonstrate superior detection accuracy, reduced false positives, resistance against mimicry attacks, and significant improvements in scalability compared to existing provenance-based IDS systems.

Contents

1	Introduction	3
2	Provenance Graphs	3
2.1	Components of Provenance Graph	3
2.2	Example	3
3	FLASH Architecture	4
4	Working of FLASH IDS	4
4.1	Step 1: Log Collection	4
4.2	Step 2: Provenance Graph Construction	4
4.3	Step 3: Semantic Encoding	4
4.3.1	Attributes Used	5
4.4	Step 4: Temporal Encoding	5
4.4.1	Positional Encoding Formula	5
5	Graph Neural Network (GNN)	5
5.1	Why GNN?	5
5.2	GraphSAGE Aggregation	5
6	Selective Graph Traversal	5
6.1	Traversal Principles	6
7	Embedding Recycling Database	6
7.1	Motivation	6
7.2	Solution	6
8	Lightweight Classifier	6
8.1	Why XGBoost?	6
8.2	Detection Logic	7
9	Attack Scenario and Detection	7
9.1	Attack Flow	7
9.2	Example Attack Graph	7

10 Attack Evolution Graphs (AEG)	7
10.1 Benefits	8
11 Performance Evaluation	8
11.1 Datasets Used	8
11.2 Performance Metrics	8
11.3 Results	8
12 Advantages of FLASH IDS	8
13 Limitations	9
14 Future Work	9
15 Conclusion	9

1 Introduction

Cybersecurity threats have evolved significantly over the past decade. Advanced Persistent Threats (APTs) have become one of the most dangerous forms of attacks due to their stealthy behavior, persistence, and ability to bypass traditional security systems.

Traditional Intrusion Detection Systems (IDS) primarily depend upon:

- Signature matching
- Rule-based detection
- Statistical anomaly detection

However, these techniques suffer from several limitations:

- Inability to detect zero-day attacks
- High false positive rates
- Poor contextual understanding
- Weak scalability in enterprise systems

To address these issues, provenance-based intrusion detection systems have emerged as a promising solution. FLASH IDS is one such advanced framework that combines:

- Provenance graphs
- Word2Vec semantic embeddings
- Temporal encoding
- Graph Neural Networks
- Embedding recycling
- Lightweight machine learning classifiers

FLASH enables accurate and scalable real-time intrusion detection.

2 Provenance Graphs

A provenance graph represents causal relationships between system entities and activities.

2.1 Components of Provenance Graph

Table 1: Provenance Graph Components

Component	Description
Process Nodes	Running applications/processes
File Nodes	Files accessed by processes
Socket Nodes	Network communication endpoints
Module Nodes	Libraries and DLLs loaded
Edges	System calls and interactions

2.2 Example

```
Chrome.exe --> downloads malware.exe
malware.exe --> reads password.txt
malware.exe --> sends data to attacker IP
```

These interactions form a graph structure that captures attack causality and system behavior.

3 FLASH Architecture

FLASH IDS consists of the following major components:

1. Provenance Graph Constructor
2. Semantic Encoder
3. Temporal Encoder
4. Graph Neural Network Encoder
5. Embedding Recycling Database
6. Lightweight Classifier
7. Attack Evolution Graph Generator

Figure 1: FLASH IDS Architecture

4 Working of FLASH IDS

4.1 Step 1: Log Collection

FLASH collects:

- Windows Event Logs
- Linux Audit Logs
- File activities
- Process executions
- Network connections

4.2 Step 2: Provenance Graph Construction

The collected logs are transformed into provenance graphs where:

- Nodes represent system entities
- Edges represent interactions

4.3 Step 3: Semantic Encoding

FLASH uses Word2Vec to convert semantic attributes into dense vectors.

4.3.1 Attributes Used

Table 2: Semantic Attributes

Node Type	Attributes
Process	Process name, command line
File	File path
Socket	IP address, port
Module	Module path

4.4 Step 4: Temporal Encoding

FLASH preserves temporal order using positional encoding.

4.4.1 Positional Encoding Formula

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

This helps the system understand event sequences.

5 Graph Neural Network (GNN)

FLASH employs GraphSAGE to learn graph representations.

5.1 Why GNN?

GNN helps:

- Learn node relationships
- Capture local and global graph structures
- Understand neighborhood behavior

5.2 GraphSAGE Aggregation

$$h_v^{(k)} = \sigma\left(W^{(k)} \cdot \text{AGGREGATE}(h_u^{(k-1)} : u \in N(v))\right)$$

Where:

- $N(v)$ = Neighbor set
- W = Learnable weight matrix
- σ = Activation function

6 Selective Graph Traversal

To improve scalability, FLASH introduces selective traversal principles.

6.1 Traversal Principles

1. Unique Edge Sampling
2. Low Priority Event Exclusion
3. Execution-Specific Filtering
4. User-Specific Attribute Handling

These optimizations significantly reduce graph complexity.

7 Embedding Recycling Database

One of FLASH’s major innovations is the embedding recycling database.

7.1 Motivation

Running GNN continuously on enterprise-scale graphs is computationally expensive.

7.2 Solution

FLASH stores previously computed embeddings.

Table 3: Embedding Database

Persistent Node ID	Stored Embedding
chrome.exe	Vector Representation
powershell.exe	Vector Representation
explorer.exe	Vector Representation

During runtime:

- Existing embeddings are reused
- GNN computation is avoided
- Real-time scalability improves

8 Lightweight Classifier

FLASH uses XGBoost for fast anomaly detection.

8.1 Why XGBoost?

- High speed
- High accuracy
- Scalable
- Efficient inference

8.2 Detection Logic

The classifier predicts node types using:

- Word2Vec embeddings
- GNN embeddings

Misclassified nodes are treated as anomalies.

9 Attack Scenario and Detection

9.1 Attack Flow

1. Phishing email delivery
2. PowerShell execution
3. Command & Control connection
4. File reconnaissance
5. Malware download
6. Data compression
7. Data exfiltration

9.2 Example Attack Graph

```
Outlook.exe
  ↓
powershell.exe
  ↓
find.exe
  ↓
sensitive files
  ↓
zip.exe
  ↓
export.zip
  ↓
attacker server
```

FLASH identifies:

- Suspicious sequences
- Abnormal graph structures
- Malicious semantic behavior

10 Attack Evolution Graphs (AEG)

FLASH generates compact attack evolution graphs to assist analysts.

10.1 Benefits

- Simplified investigation
- Faster attack reconstruction
- Reduced analyst workload
- Better attack visualization

11 Performance Evaluation

11.1 Datasets Used

- DARPA OpTC
- DARPA E3
- StreamSpot
- Unicorn

11.2 Performance Metrics

Table 4: Performance Metrics

Metric	Description
Precision	Correct positive predictions
Recall	Detection capability
F-score	Combined accuracy measure
False Positives	Incorrect alerts
Inference Time	Detection speed

11.3 Results

FLASH demonstrates:

- Higher precision
- Lower false positives
- Faster inference
- Better scalability

12 Advantages of FLASH IDS

Table 5: Advantages

Feature	FLASH IDS
Semantic Understanding	Yes
Temporal Understanding	Yes
Graph Learning	Yes
Real-time Detection	Yes
Low False Positives	Yes
Mimicry Attack Resistance	Yes
Scalable Architecture	Yes

13 Limitations

Despite its advantages, FLASH has some limitations:

- Dependency on provenance logs
- Large storage requirements
- Complex deployment in enterprises
- High training complexity

14 Future Work

Possible future improvements include:

- Transformer-based graph learning
- Integration with LLMs
- Automated response systems
- Cross-host attack correlation
- Cloud-native deployment

15 Conclusion

FLASH IDS presents a powerful and scalable approach for detecting sophisticated cyber attacks using provenance graph representation learning. By combining semantic understanding, temporal encoding, Graph Neural Networks, and embedding recycling, FLASH achieves superior detection performance while maintaining real-time scalability.

The system successfully addresses key challenges faced by traditional intrusion detection systems and demonstrates strong resistance against stealthy Advanced Persistent Threats.

References

- [1] Mati Ur Rehman et al., *FLASH: A Comprehensive Approach to Intrusion Detection via Provenance Graph Representation Learning*, IEEE Symposium on Security and Privacy, 2024.
- [2] Hamilton, Ying, and Leskovec, *Inductive Representation Learning on Large Graphs*, NeurIPS, 2017.
- [3] Mikolov et al., *Efficient Estimation of Word Representations in Vector Space*, 2013.
- [4] Chen and Guestrin, *XGBoost: A Scalable Tree Boosting System*, KDD, 2016.